

Technical Documentation: Portfolio E-Commerce System

1. Overview

This repository contains a **full-stack e-commerce application** built using **JavaScript**, **Node.js**, **Express**, **MongoDB**, and **React**. The system is structured into multiple modules, including:

- Frontend (React + Next.js):** Admin dashboard and user-facing storefront.
- Backend (Express):** RESTful API for handling business logic, authentication, and database operations.
- Database (MongoDB):** Stores user data, products, orders, and other entities.
- Validation (Zod):** Ensures data integrity via schema validation.

2. Architecture

The system follows a **modular monorepo** structure with distinct directories for each component:

```
portfolio/
├── ecommerce/
│   ├── admin-frontend/    # Next.js admin dashboard
│   ├── api/               # Express backend API
│   └── frontend/          # Next.js storefront
├── mern-contact/         # Legacy contact form (MERN stack)
└── package.json          # Root-level dependencies
```

Tech Stack

Component	Technology Stack	Frontend	Next.js (React), TailwindCSS, TypeScript
Backend	Express.js, Mongoose (MongoDB ODM)	Database	MongoDB (Atlas or local)
Validation	Zod	Authentication	JWT (JSON Web Tokens)
Testing	Jest, Supertest		

3. Setup & Installation

Prerequisites

- Node.js (v18+)
- MongoDB (local or Atlas)

- Docker (optional, for MongoDB)

Installation Steps

1. Clone the Repository

```
git clone <repository-url>
cd portfolio
```

2. Install Dependencies

```
# Install root-level dependencies (if any)
npm install

# Navigate to the API directory and install dependencies
cd ecommerce/api
npm install

# Navigate to the admin-frontend and install dependencies
cd ../admin-frontend
npm install

# Navigate to the frontend and install dependencies
cd ../frontend
npm install
```

3. Configure Environment Variables

Create a `.env` file in the `ecommerce/api` directory with the following variables:

```
MONGO_URI=mongodb://localhost:27017/ecommerce
JWT_SECRET=your_jwt_secret_key
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_email_password
PORT=3000
```

4. Run the Application

- **Backend (API)**

```
cd ecommerce/api
npm run dev
```

The API will start on `http://localhost:3000`.

- **Admin Frontend**

```
cd ../admin-frontend
npm run dev
```

The admin dashboard will start on `http://localhost:3000`.

- **Storefront (Frontend)**

```
cd ../frontend
npm run dev
```

The storefront will start on `http://localhost:3000`.

4. API Documentation

Base URL

```
http://localhost:3000/api/v1
```

Authentication

- **JWT Authentication:** All protected routes require a valid JWT token in the `Authorization` header.

```
Authorization: Bearer <token>
```

- **Authentication Middleware:** Located in `ecommerce/api/middleware/auth.js`.

Routes

Users

Method	Endpoint	Description	Auth Required						
				-----	-----	-----	-----	-----	-----

					POST	<code>/users/register</code>	Register a new user	No	
					POST	<code>/users/login</code>	Login a user	No	
					POST	<code>/users/logout</code>	Logout a user	Yes	
					POST	<code>/users/forgot-password</code>	Initiate password reset	No	
					GET	<code>/users</code>	List all users (admin only)	Yes (Admin)	
					GET	<code>/users/:id</code>	Get a user by ID	Yes	
					PUT	<code>/users/:id</code>	Update a user	Yes (Admin)	
					DELETE	<code>/users/:id</code>	Delete a user (soft delete)	Yes (Admin)	

Products

Method	Endpoint	Description	Auth Required						
				-----	-----	-----	-----	-----	-----

					POST	<code>/products</code>	Create a new product	Yes (Admin)	
					GET	<code>/products</code>	List all products	Yes	
					GET	<code>/products/:id</code>	Get a product by ID	Yes	
					PUT	<code>/products/:id</code>	Update a product	Yes (Admin)	
					DELETE	<code>/products/:id</code>	Delete a product (soft delete)	Yes (Admin)	

Orders

Method	Endpoint	Description	Auth Required						
				-----	-----	-----	-----	-----	-----

					POST	<code>/orders</code>	Create a new order	Yes	
					GET	<code>/orders</code>	List all orders	Yes	
					GET	<code>/orders/:id</code>	Get an order by ID	Yes	
					PUT	<code>/orders/:id</code>	Update an order	Yes (Admin)	
					DELETE	<code>/orders/:id</code>	Delete an order (soft delete)	Yes (Admin)	

Invoices

Method	Endpoint	Description	Auth Required	
POST	<code>/invoices</code>	Create a new invoice	Yes (Admin)	
GET	<code>/invoices</code>	List all invoices	Yes	
GET	<code>/invoices/:id</code>	Get an invoice by ID	Yes	
POST	<code>/invoices/pay</code>	Pay an invoice	Yes (Admin)	
DELETE	<code>/invoices/:id</code>	Delete an invoice (soft delete)	Yes (Admin)	

Carts

Method	Endpoint	Description	Auth Required	
POST	<code>/carts</code>	Add a product to cart	Yes	
GET	<code>/carts/:id</code>	Get a user's cart	Yes	
DELETE	<code>/carts/items/:productId</code>	Remove a product from cart	Yes	
DELETE	<code>/carts/clear</code>	Clear the cart	Yes	

Geographies (States & Cities)

Method	Endpoint	Description	Auth Required	
POST	<code>/geographies/states</code>	Register a new state	Yes (Admin)	
POST	<code>/geographies/cities</code>	Register a new city	Yes (Admin)	
GET	<code>/geographies/states</code>	List all states	Yes	
GET	<code>/geographies/cities</code>	List all cities	Yes	

Emails

Method	Endpoint	Description	Auth Required	
POST	<code>/emails/create</code>	Create a new email record	Yes (Admin)	
POST	<code>/emails/send/bulk</code>	Send bulk emails	Yes (Admin)	

5. Database Schema

MongoDB Collections

Collection	Fields	Description	
User	<code>email</code> , <code>password</code> , <code>firstname</code> , <code>lastname</code> , <code>businessName</code> , <code>phone</code> , <code>addresses</code> , <code>type</code> , <code>deleted</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores user information.	
Product	<code>name</code> , <code>description</code> , <code>price</code> , <code>category</code> , <code>quantity</code> , <code>images</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores product details.	
Order	<code>userId</code> , <code>products</code> , <code>status</code> , <code>shippingAddress</code> , <code>paymentMethod</code> , <code>paymentStatus</code> , <code>trackingNumber</code> , <code>totalAmount</code> , <code>deliveryFee</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores order information.	
Invoice	<code>userId</code> , <code>orderId</code> , <code>invoiceNumber</code> , <code>totalAmount</code> , <code>status</code> , <code>products</code> , <code>proofOfPayment</code> , <code>discountAmount</code> , <code>discountReason</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores invoice details.	
Cart	<code>userId</code> , <code>products</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores user carts.	
Category	<code>name</code> , <code>description</code> , <code>parentCategory</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>deletedAt</code>	Stores product categories.	
State	<code>name</code> , <code>code</code> , <code>cities</code> , <code>createdAt</code>	Stores state information.	
City	<code>name</code> , <code>stateId</code> , <code>coordinates</code> , <code>createdAt</code>	Stores city information.	
Email	<code>to</code> , <code>subject</code> , <code>body</code> , <code>sentAt</code> , <code>status</code>	Stores email records.	

6. Configuration

Environment Variables

Variable	Description	Example Value			
<code>MONGO_URI</code>	MongoDB connection URI	<code>mongodb://localhost:27017/ecommerce</code>	<code>JWT_SECRET</code>	JWT secret key	<code>your_jwt_secret_key</code>
<code>EMAIL_USER</code>	Email for sending notifications	<code>your_email@gmail.com</code>	<code>EMAIL_PASS</code>	Email password	<code>your_email_password</code>
<code>PORT</code>	API port	<code>3000</code>			

Validation Schemas (Zod)

Validation schemas are defined in `ecommerce/api/utils/schemas/` to ensure data integrity. Examples include:

- `userSchemas.js`: Validates user registration/login data.
- `productSchema.js`: Validates product creation/update data.
- `orderSchema.js`: Validates order creation/update data.

7. Development Guidelines

Code Style

- EditorConfig**: Enforced via `.editorconfig` (4-space indentation, UTF-8 encoding).
- TypeScript**: Used for type safety in the frontend.
- ESLint**: Configured for Next.js projects in `admin-frontend` and `frontend`.

Testing

- Backend Testing**: Uses Jest and Supertest for API testing.
- Test Setup**: MongoDB Memory Server for in-memory testing.
- Test Files**: Located in `ecommerce/api/tests/`.

Best Practices

- Separation of Concerns**: Controllers handle business logic; models manage database operations.
- Authentication**: JWT-based with role-based access control (RBAC).
- Error Handling**: Centralized error handling middleware (`ecommerce/api/middleware/errorHandler.js`).
- Validation**: Zod schemas for request/response validation.
- Soft Deletes**: Uses `deletedAt` fields instead of hard deletes.

8. Deployment Instructions

Deploying the Backend (API)

1. Dockerize the API (optional):

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Build and run:

```
docker build -t ecommerce-api .
docker run -p 3000:3000 ecommerce-api
```

2. Deploy to a Cloud Provider (e.g., Vercel, Render, or AWS):

- Use **Vercel** for the Next.js frontend.
- Use **Render** or **Heroku** for the Express backend.

Deploying the Frontend (Next.js)

1. Vercel Deployment:

- Push to a Git repository (GitHub/GitLab).
- Import the project into Vercel.
- Vercel will automatically deploy the Next.js app.

2. Manual Deployment:

```
npm run build
npm run start
```

9. Troubleshooting

Common Issues & Fixes

Issue	Solution
MongoDB Connection Errors	Ensure <code>MONGO_URI</code> is correct and MongoDB is running.
JWT Authentication Errors	Verify <code>JWT_SECRET</code> and token generation logic.
CORS Errors	Ensure <code>cors()</code> middleware is configured correctly.

in Express. || **Frontend Not Loading** | Check if the backend API is running and accessible from the frontend. || **Validation Errors** | Review Zod schemas and ensure request data matches expected formats. |

10. Future Improvements

1. **Add Payment Gateway Integration** (Stripe, PayPal).
 2. **Implement Caching** (Redis) for frequently accessed data.
 3. **Add Analytics** (Google Analytics, custom tracking).
 4. **Improve UI/UX** (Dark mode, better error messages).
 5. **Add Unit/Integration Tests** for critical paths.
-

11. Contacts

For questions or support, contact:

- **Developer:** Jamilah Bello
 - **Email:** jamilahbello@outlook.com
 - **GitHub:** <https://github.com/JamilahBello>
-

This documentation provides a comprehensive guide to setting up, running, and extending the **portfolio e-commerce system**.